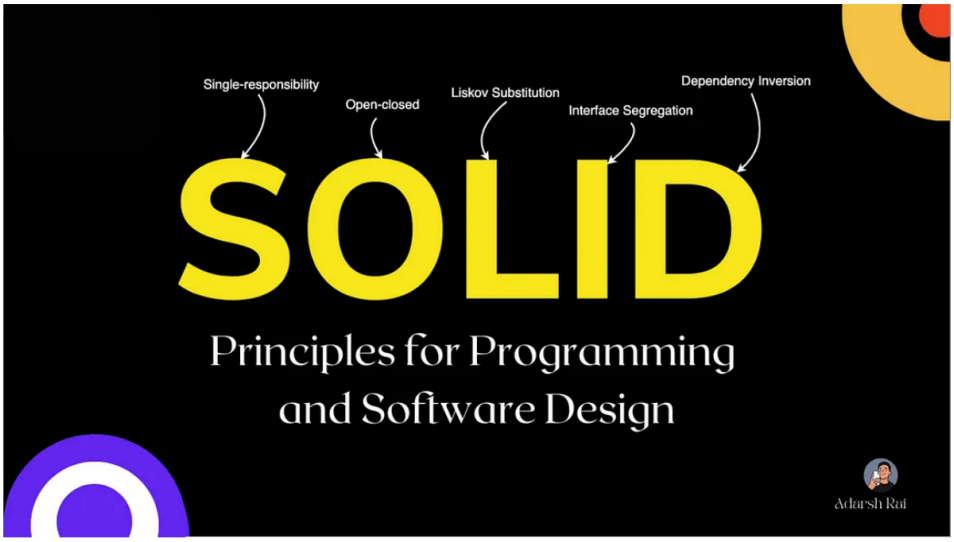


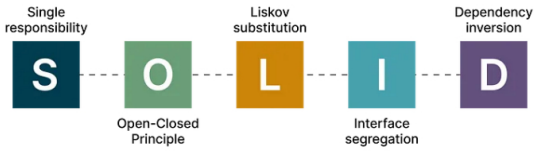
Mastering SOLID Principles in JavaScript: A Comprehensive Guide

Adarsh Rai · Follow
7 min read · Dec 18, 2023

SOLID principles are closely linked to design patterns, which are crucial knowledge, especially in an interview context. Familiarity with these principles not only demonstrates expertise but also aids in comprehending advanced programming paradigms, architectural patterns, and language features like reactive programming, flux architecture (Redux), JavaScript generators, and more.

What exactly are the SOLID principles?

SOLID is an acronym representing:



These five principles serve as guiding pillars for crafting superior code. Although they originate from object-oriented programming, it's quite audacious to categorize JavaScript solely as an object-oriented language! Nevertheless, understanding these principles will prompt critical thinking while designing solutions. You'll frequently ask yourself, "Am I breaching the Single Responsibility Principle?" when formulating your next solution.

So, let's delve into these principles. In this article, we'll explore applying these principles in JavaScript and illustrate them using practical code samples.

Understanding the Single Responsibility Principle

The Single Responsibility Principle (SRP) suggests that a class, module, or function should serve just one role or responsibility. In simpler terms, it should have a single reason to change.

Let's consider an example to grasp this concept. Imagine a JavaScript class named ManageEmployee with functions for various employee management tasks:

```
  constructor(private http: HttpClient)
  SERVER_URL = 'http://localhost:5000/employee';

  getEmployee(empId) {
    return this.http.get(this.SERVER_URL + `/${empId}`);
  }

  updateEmployee(employee) {
    return this.http.put(this.SERVER_URL + `/${employee.id}`, employee);
  }

  deleteEmployee(empId) {
    return this.http.delete(this.SERVER_URL + `/${empId}`);
  }

  calculateEmployeeSalary(empId, workingHours) {
    var employee = this.http.get(this.SERVER_URL + `/${empId}`);
    return employee.rate * workingHours;
  }
}
```

At first glance, this code might seem alright, but it actually violates the Single Responsibility Principle. The `ManageEmployee` class handles two different responsibilities: HR management (get, update, delete) and finance management (calculate salary).

If you need to modify a function related to HR or finance in the future, changes in the `ManageEmployee` class will impact both areas. Therefore, it's essential to separate functionalities concerning HR and finance departments to align with the Single Responsibility Principle.

Here's an adjusted code example achieving this separation:

Adarsh.js

```
class ManageEmployee {
  constructor(private http: HttpClient)
  SERVER_URL = 'http://localhost:5000/employee';

  getEmployee(empId) {
    return this.http.get(this.SERVER_URL + `/${empId}`);
  }

  updateEmployee(employee) {
    return this.http.put(this.SERVER_URL + `/${employee.id}`, employee);
  }

  deleteEmployee(empId) {
    return this.http.delete(this.SERVER_URL + `/${empId}`);
  }
}

class ManageSalaries {
  constructor(private http: HttpClient)
  SERVER_URL = 'http://localhost:5000/employee';

  calculateEmployeeSalary(empId, workingHours) {
    var employee = this.http.get(this.SERVER_URL + `/${empId}`);
    return employee.rate * workingHours;
  }
}
```

By dividing functionalities into separate classes (`ManageEmployee` and `ManageSalaries`), we adhere to the Single Responsibility Principle, ensuring each class is responsible for one specific aspect of the application.

Open-closed principle

The Open-Closed Principle asserts that functions, modules, and classes ought to be extendable without necessitating modifications. This principle holds paramount importance when implementing large-scale applications. It emphasizes the ability to easily introduce new features without altering existing code, preventing disruptive changes.

For instance, consider a scenario where a function called `calculateSalaries()` computes salaries based on predefined job roles and hourly rates stored in an array:

Adarsh.js

```
class ManageSalaries {
  constructor() {
    this.salaryRates = [
      { id: 1, role: 'developer', rate: 100 },
      { id: 2, role: 'architect', rate: 200 },
      { id: 3, role: 'manager', rate: 300 },
    ];
  }

  calculateSalaries(empId, hoursWorked) {
    let salaryObject = this.salaryRates.find((o) => o.id === empId);
    return hoursWorked * salaryObject.rate;
  }
}
```

Directly altering the `salaryRates` array contradicts the Open-Closed Principle. For instance, if you need to incorporate a new role into salary calculations, it's crucial to extend the code without modifying the original structure. To align with this principle, a separate method like `addSalaryRate()` can be introduced to append new salary rates:

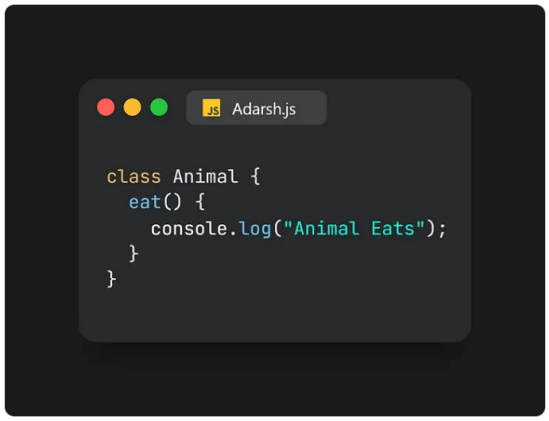
By employing `addSalaryRate()` to include new rates, the existing code remains unchanged, aligning with the Open-Closed Principle. This approach facilitates extending functionality without modifying the original codebase.

Liskov substitution principle

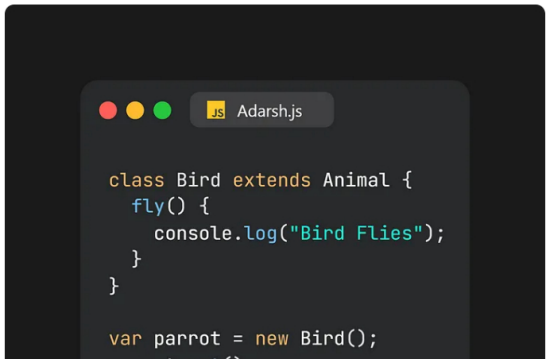
The Liskov Substitution Principle (LSP) can be stated as follows: If P(y) is a property true about objects y of type A, then P(x) should also hold true for objects x of type B, where B is a subtype of A.

While various explanations exist online, they all convey the same essence. Put simply, the Liskov principle dictates that substituting a parent class with its subclasses should not cause unexpected behaviors in the application.

For instance, consider an Animal class with an eat() function:



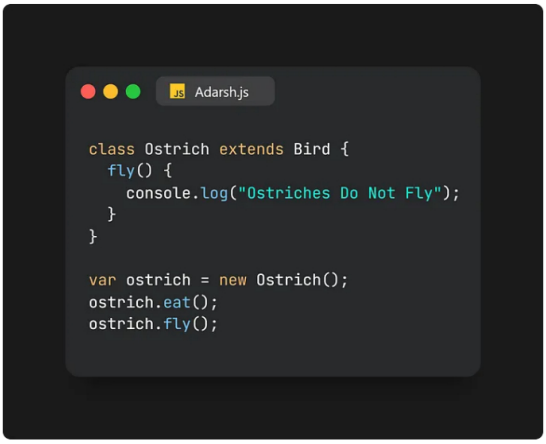
Extending this class to Bird with a fly() function:



```
parrot.eat();
parrot.fly();
```

In this example, creating an object parrot from the Bird class and calling both eat() and fly() methods does not violate the Liskov principle. The parrot can perform both actions, aligning with expectations.

However, extending Bird further to create an Ostrich class:



This violates the Liskov principle because ostriches cannot fly, potentially causing unexpected behavior. The optimal approach is to extend the Ostrich class from the Animal class instead:

By inheriting from the Animal class rather than Bird, the Ostrich class adheres to the Liskov Substitution Principle, ensuring expected behavior in the application.

Interface segregation principle

The Interface Segregation Principle advocates that clients should not be compelled to depend on interfaces they won't utilize. It emphasizes breaking down extensive interfaces into smaller, more specific ones. To illustrate this, consider attending a driving school where you're handed comprehensive instructions covering driving cars, trucks, and trains. If you're there to solely learn car driving, you don't need the excess information about trucks and trains. Hence, the school should provide instructions tailored to your specific need — driving cars.

Although JavaScript lacks native support for interfaces, implementing this principle in JavaScript-based applications can be challenging. However, developers can leverage JavaScript compositions to achieve a similar effect. Compositions allow adding functionalities to a class without inheriting the entire class.

For instance, in the given example, there's a class called DrivingTest containing methods like startCarTest and startTruckTest. Extending this class for CarDrivingTest and TruckDrivingTest forces both classes to implement functions for both cars and trucks.

To align with the Interface Segregation Principle, we can use compositions to attach required functionalities specifically to the relevant classes, as demonstrated in the following code snippet:

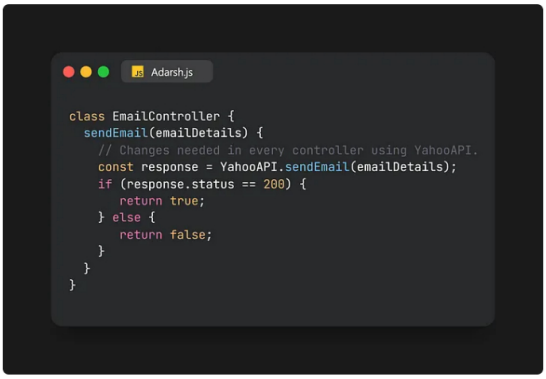
This implementation adheres to the Interface Segregation Principle as functionalities are assigned only to the relevant classes, ensuring that unnecessary functionalities aren't enforced upon them.

Dependency inversion principle

The Dependency Inversion Principle revolves around the idea that higher-level modules should rely on abstractions rather than directly on lower-level modules. This principle aims to untangle code dependencies, offering the flexibility to modify and expand applications at higher levels without encountering complications.

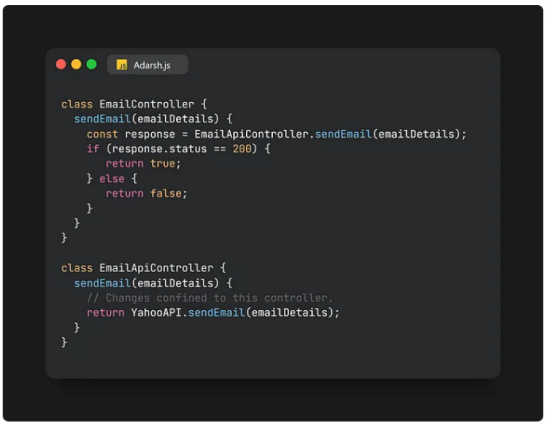
In the context of JavaScript, the concept of abstractions is nuanced due to the language's dynamic nature. However, it remains crucial to ensure that higher-level modules don't establish dependencies on lower-level modules.

Allow me to illustrate how the Dependency Inversion Principle operates through a simple example. Suppose your application initially utilized the Yahoo email API, and now there's a need to transition to the Gmail API. If you've built controllers without adhering to dependency inversion, such as the example below, updating to the new API requires alterations across multiple controllers entwined with the Yahoo API.



Implementing the Dependency Inversion Principle helps avert these

cumbersome modifications. By segregating the email API handling into a distinct controller, updates to the email API necessitate changes solely within that dedicated controller.



Applying this principle simplifies maintenance and updates, as modifications related to the email API are confined to a specific, designated controller, offering more manageable and less error-prone changes across the application.

Implementing the Dependency Inversion Principle can significantly enhance the maintainability and scalability of software systems. By adhering to this principle, developers create a more loosely-coupled architecture, allowing for seamless modifications and updates at higher abstraction levels without necessitating extensive changes throughout the codebase.

In embracing a mindset that prioritizes abstractions over direct dependencies, teams gain the agility to adapt and incorporate new functionalities or switch underlying components without causing cascading disruptions. This practice fosters code that is more flexible, resilient, and easier to manage in the face of evolving requirements or technological advancements.

Ultimately, applying the Dependency Inversion Principle not only streamlines the development process but also contributes to the creation of robust, adaptable software solutions poised to meet the dynamic demands of the ever-evolving technological landscape.

. . .

In conclusion

The SOLID principles — Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion — serve as foundational pillars guiding software development toward robustness, maintainability, and scalability.

These principles offer a roadmap for crafting clean, modular, and adaptable code. By emphasizing concepts like cohesive functionality, extensibility without modification, substitutability of objects, interface clarity, and abstraction over concrete dependencies, SOLID principles pave the way for more resilient and future-proof software systems.

Adhering to SOLID principles fosters code that is easier to understand, modify, and extend. It encourages the creation of software architectures that can readily accommodate changes, scale efficiently, and withstand the test of time. Embracing these principles doesn't just elevate the quality of code; it cultivates a development mindset geared towards crafting sustainable solutions capable of meeting evolving needs in the ever-dynamic realm of software engineering.

> *The value of SOLID principles is not obvious. But if you ask yourself "Am I violating SOLID principles" when you design your architecture, I promise that the quality and scalability of your code will be much better.*

Thanks a lot for reading!
Feel free to follow me here on Medium.com and also on Twitter (@adarshrai00)

Peace!

JavaScript | Javascript Tips | Web Development | Programming | Software Development

195 1

Written by Adarsh Rai

Follow

849 Followers

Passionate Developer and Tech Enthusiast | Sharing Insights on React, JavaScript, Web Development, and MERN Stack | Code Improvement Advocate

More from Adarsh Rai

Adarsh Rai

Web Development Trends For 2024

The ever-shifting landscape of digital innovation can feel like a relentless race, a...

15 min read · Feb 13, 2024

549 9

Adarsh Rai

53 JavaScript Frontend Interview Questions

Introduction

19 min read · Jan 13, 2024

682 10

Adarsh Rai

Uncover the JavaScript Interview Question That Stumps 90% of...

In the realm of JavaScript interviews, there are questions that appear simple at first but...

10 min read · Mar 28, 2024

84 3

Adarsh Rai

JavaScript Secrets Revealed: 21 Tricks Every Coder Should Master!

Welcome to the ultimate JavaScript revelation! Uncover 21 powerful coding...

7 min read · Nov 22, 2023

509 9

See all from Adarsh Rai

Recommended from Medium

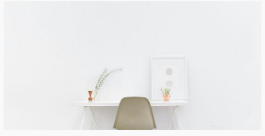
Pinjari Rehan in JavaScript in Plain English

15 Time-Saving Websites Every Developer Needs

Ever thought that there aren't enough hours in the day for all your never-ending...

5 min read · Apr 29, 2024

731 9



Aashish Peepra

How to design clean API interfaces

This is going to be a tough read. This contains too much code to read and no pictures. It's...

7 min read · Mar 3, 2024

439 5

Lists

General Coding Knowledge

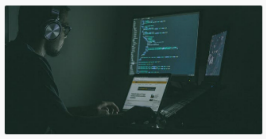
20 stories · 1200 saves

Stories to Help You Grow as a Software Developer

19 stories · 1045 saves

Coding & Development
11 stories · 601 saves

Leadership
50 stories · 324 saves



Mate Marschalko

28 JavaScript One-Liners every Senior Developer Needs to Know

Learn how to implement complex logic with beautifully short and efficient next-level...

· 9 min read · Mar 1, 2024

291 2

Tari Ibaba in Coding Beauty

These are the 5 most transformative JavaScript feature...

5 juicy ES9 features that transformed JavaScript with new functionality and cleane...

· 5 min read · May 2, 2024

183 1

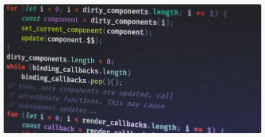
Prakash Pun

Javascript Memoization

What is Memoization ?

3 min read · Feb 19, 2024

72 1



Khushi_developer

Avoiding Unnecessary Re-renders: Common Mistakes in React...

React is renowned for its performance, but as developers, it's crucial to be mindful of...

2 min read · Dec 14, 2023

117 7

See more recommendations